Dynamic Programming

Dynamic Programming and Deep Learning

Mikkel Reich

Department of Economics, University of Copenhagen

Plan for today

- A bit about my own journey in Deep Learning
- Motivation: Curse of Dimensionality
- New tools: Reinforcement and Deep Learning
- A Deep Learning algorithm: DeepSimulate (Druedahl & Røpke 2026)
- Solving a simple model using DeepSimulate (+ code)
- Ideas for term papers

Why does your TA know about Deep Learning?

- Bachelor Thesis: *Examining a Method from Deep Reinforcement Learning as an Alternative to Value Function Iteration*
  - Model with 3D state space
  - Solved model with both VFI and DeepSimulate (or "DeepExplore")
  - Check code + PDF at github repo
- Research Assistant on project: *Modelling Economic Agents as Deep Reinforcement Learners*
  - Multiple algorithms for both convex and non-convex models with increasing complexity
  - User-friendly Python package: EconDLSolvers (github repo)
  - Multiple working papers, see Jeppes website

Motivation:

Curse of Dimensionality

Why do we need Deep and/or Reinforcement Learning?

Central bottleneck in Dynamic Programming algorithms:

Curse of Dimensionality (CD)

We solve models on grids of state variables.

$\rightarrow$ Optimal policy <u>for each combination</u> of state variable grid points

$\rightarrow$ Solve model on "tensor product grid" / "Cartesian grids"

Clear CD in #states (our main focus today)

Quadrature $\rightarrow$ CD in #shocks

Grid search for optimization $\rightarrow$ CD in #choices

Curse of Dimensionality in action

Consider model with n states and 50 grid points for each state.

Value and policy functions defined on n-dimensional grid.

Eventually ..., we will encounter nested loops:

For $state_1$ in $state_1$-grid:

    For $state_2$ in $state_2$-grid:

        ... For $state_n$ in $state_n$-grid:

            Interpolate next-period policy/value, compute expectation, optimize, ... etc.

Bottom-line: We have to do 50^n operations

Curse of Dimensionality: Perspectives

$n = 3 \Rightarrow \#\text{gridpoints} = 50^3 = 125\ 000$

$n = 5 \Rightarrow \#\text{gridpoints} = 50^5 = 312\ 500\ 000$

$n = 10 \Rightarrow \#\text{gridpoints} = 50^{10} \approx 9.765 * 10^{16}$

More than a couple of states $\rightarrow$ You will feel it!

Will we ever have more than "a couple of states"?

Definition of state variable: Information needed to make decision

For Consumption-Saving models:

- Income (persistent vs. transitory)
- Unemployment
- Return on saving (portfolio choice: risky vs. safe assets)

Can "explode" in e.g. General Equilibirum

New tools: Reinforcement and Deep Learning

Reinforcement and Deep Learning

Recent literature use methods from Deep Learning and Reinforcement Learning to avoid CD by:

1. Avoiding grids $\rightarrow$ Use simulation
2. Using Deep Neural Networks (DNN) for function approximation

Deep Learning $\approx$ Training DNNs

Reinforcement Learning (RL) $\approx$ Learn optimal decisions

- Large similarities between RL and DP
- RL often assumes only partially known environment

  $\rightarrow$ Methods today are more "structural RL"

Deep Neural Networks

DNNs are flexible, nonlinear functions $f(\cdot)$ of inputs x:

$$DNN(x) = f(x; w, b)$$

, parameterized by weights w and biases b.

Hornik et. al. 1989 shows DNNs are **Universal Function Approximators**

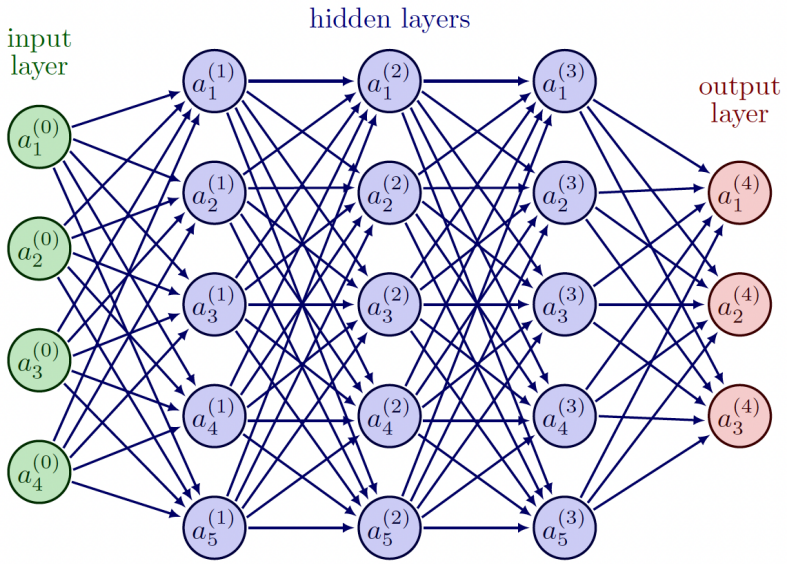
- Under certain conditions, DNNs can approximate **any** function
- Similarities to an old friend: Polynomials

DNN $\approx$ Think of big polynomial with <u>many parameters!</u>

$+$ We have great software to optimize wrt. parameters

$+$ Algorithms can be massively speed up on GPUs

If you google Deep Neural Networks



- Feed-forward DNN with 3 hidden layers
- Node $a_i^{(j)}$ represents neuron i for layer j
- Inputs are $a_i^{(0)},\ i = 1, 2, 3, 4$ and outputs are $a_i^{(4)},\ i = 1, 2, 3$
- Neurons (except inputs) has value (where $\sigma(\cdot)$ is activation fun.):

$$a_i^{(j)} = \sigma\left(b_i^{(j)} + \sum_{k=1}^{\#\text{neurons in layer } j-1} w_{k,i}^{(j)} a_k^{(j-1)}\right)$$

Training DNNs: Intuition in Deep Learning

Suppose DNN $f(x;\theta)$ (with $w, b \subset \theta$) we want to *fit to something*

DNN gets input x, output should be *something*

1st task: def' (+ compute) objective for *fitting something* (= loss)

2nd task: compute gradients of loss wrt. θ: $\nabla_\theta Loss$

3rd task: update θ to *fit better* (= minimize loss)

SGD (Stochastic Gradient Descent): $\theta^{new} = \theta^{old} - \alpha \nabla_\theta Loss(\theta^{old})$

Is often ADAM (Adaptive Moment Estimation)

1st and 3rd task are "easier"

2nd task? $\rightarrow$ **Automatic Differentiation**

Training DNNs: Automatic Differentiation

Automatic Differentiation: Chain rule on steroids!

Python libraries: PyTorch (today), JAX, ...

Central Idea: Differentiation follows few but clear rules

$\rightarrow$ Can always differentiate functions, altough tedious!

$\rightarrow$ Let computer keep track of "the chain" and diff'

Implication: 2nd task is easy! PyTorch computes ∇_θ automatically

Computing ∇_θ is often referred to as **backpropagation**

(See App. A.3 of my Thesis for a backpropagation algorithm using linear algebra)

A Deep Learning Algorithm: DeepSimulate

DeepSimulate: A simple application of DNNs

Problem is less about finding policy (or value) function

$\rightarrow$ Problem is grids themselves are subject to CD!

Idea: drop grids + Parametrize policy function $\pi(\cdot)$ as DNN:

$$\pi(\cdot) \approx \pi^{DNN}(\cdot\ ; \theta_\pi)$$

, where θ_π contains all weights and biases of DNN

Task at hand:

- How can we train DNN / optimize θ_π to... solve Bellman, maximize utility, think about expectations... etc. ?
- Where does the Bellman equation even fit into this framework?

Back-to-basics: Maximize a sum!

A typical first encounter with dynamic models is like:

$$U = \max_{\{c_t\}_{t=1}^{T}} \mathbb{E}\left[\sum_{t=1}^{T} \beta^t u(c_t)\right] \qquad \text{s.t. } s_{t+1} = f(s_t, c_t)$$

, where s_t are some states and c_t is some control variable

DeepSimulate solves exactly this.. directly!

1. Initialize DNN (= initialize approximator of policy function)
2. Simulate each period $t = 1, 2, \ldots, T$: Every time we have to choose $c_t \rightarrow$ Give states s_t to DNN and let it decide policy c_t
3. Compute gradients of "loss" $L = -U$ wrt. θ_π
4. Update θ_π^{new} to minimize loss (= maximize U)

Solving a simple BufferStock model using DeepSimulate

A simple BufferStock model

Consider the simple BufferStock model with Bellman Equation:

$$v_t(m_t, p_t) = \max_{a_t} u(c_t) + \beta \mathbb{E}_t[v_{t+1}(m_{t+1}, p_{t+1})]$$

$$\text{s.t.} \quad c_t = (1 - a_t) m_t$$

$$m_{t+1} = (1 + r)(m_t - c_t) + y_{t+1}$$

$$y_{t+1} = \kappa_{t+1} \psi_{t+1} p_{t+1}$$

$$p_{t+1} = \xi_{t+1} p_t^{\rho},$$

$$a_t \in [0, 1]$$

with log-normal shocks ψ_{t+1}, ξ_{t+1} + Retirement in $T^{retired}$

State-vector: $s_t = (m_t, p_t)$

Policy is savings rate $a_t = \pi_t(m_t, p_t)$ to be approximated with $\pi^{DNN}(t, m_t, p_t; \theta_\pi)$

One "episode" of DeepSimulate in BufferStock

Simulate N agents: for each agent $i = 1, 2, \ldots, N$:

1. Draw initial states $(m_{i,0}, p_{i,0})$
2. Call DNN to get policy $a_{i,0} = \pi^{DNN}(0, m_{i,0}, p_{i,0}; \theta_\pi)$
3. Compute "outcome": $c_{i,0} = (1 - a_{i,0})m_{i,0}$
4. Draw next-period shocks $(\psi_{i,1}, \xi_{i,1})$
5. Compute state-transition to get $(m_{i,1}, p_{i,1})$
6. Call DNN to get policy $a_{i,1} = \pi^{DNN}(1, m_{i,1}, p_{i,1}; \theta_\pi)$
7. Compute "outcome": $c_{i,1} = (1 - a_{i,1})m_{i,1}$
8. ... Repeat until we reach last period T

Sum over i and compute loss: $L = -U = -\sum_{i=1}^{N} \sum_{t=1}^{T} \beta^t u(c_{i,t})$

Automatic diff. $\rightarrow \nabla_\theta L \rightarrow$ update θ_π

This is one **episode**! We do many!

Under the hood in Automatic Differentiation

If we only had $N = 1$ (drop i-subscript):

$$\frac{\partial L}{\partial \theta_\pi} = -\frac{\partial a_0}{\partial \theta_\pi}\left(\frac{\partial u(c_0)}{\partial a_0} + \beta \frac{\partial m_1}{\partial a_0}\left(\frac{\partial u(c_1)}{\partial m_1} + \frac{\partial a_1}{\partial m_1}\frac{\partial u(c_1)}{\partial m_1}\right) + \beta^2 \dots\right) - \frac{\partial a_1}{\partial \theta_\pi}(\cdot)$$

DNN took decision about a_0

How did it affect utility in $t = 0$?

How did it affect state transition to $t = 1$?

How did state trans. directly affect utility in $t = 1$?

How did state trans. affect choice in $t = 1$?

$\vdots$

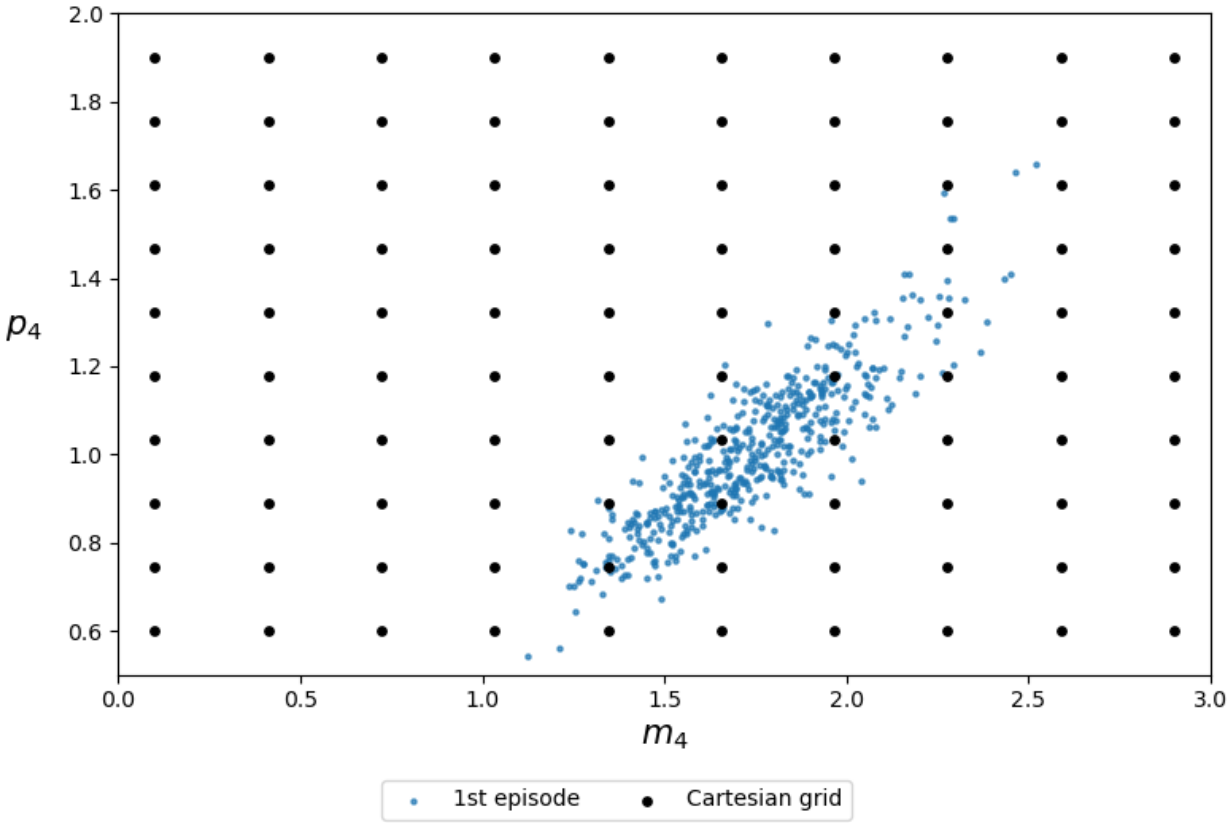
Method closely related to "policy gradient" methods

Implementation of DeepSimulate for BufferStock model in PyTorch

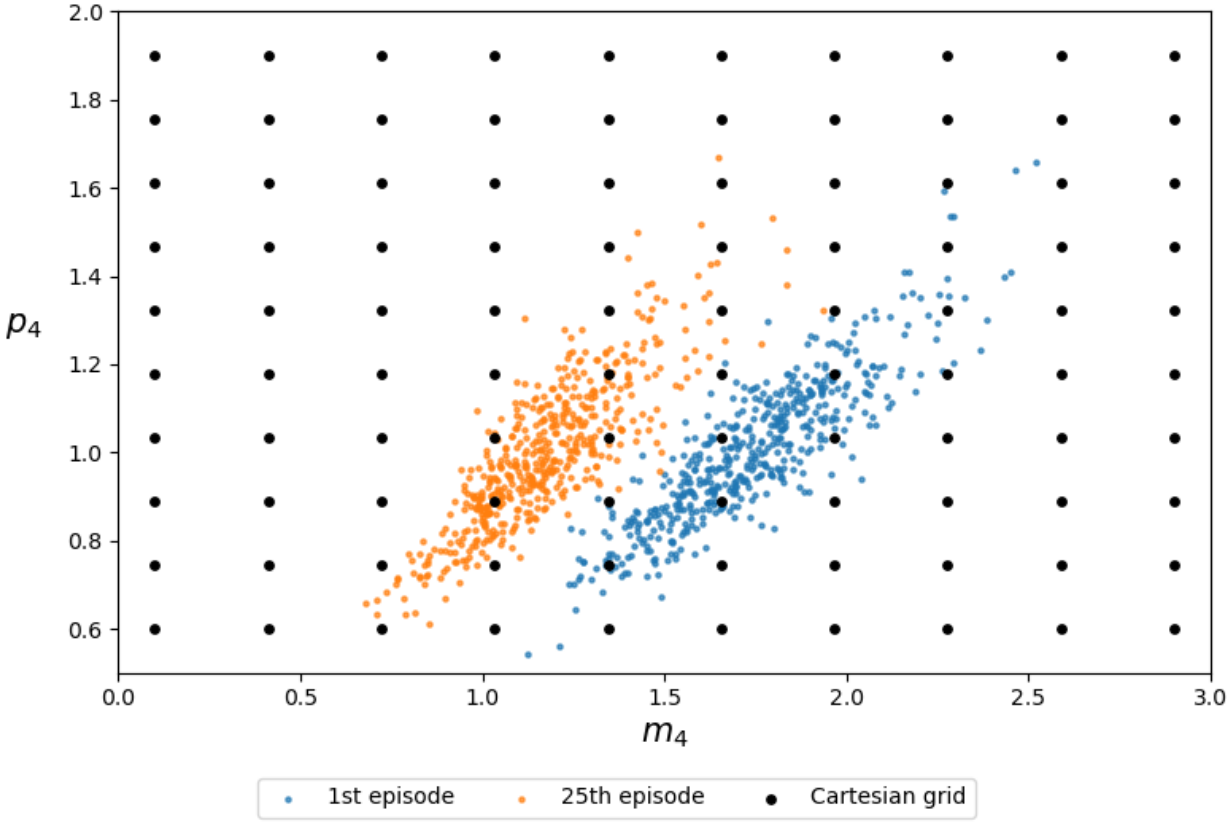
Grids vs simulation: 1st episode

Cartesian grid: the standard we always use

1st episode: No training of DNN yet

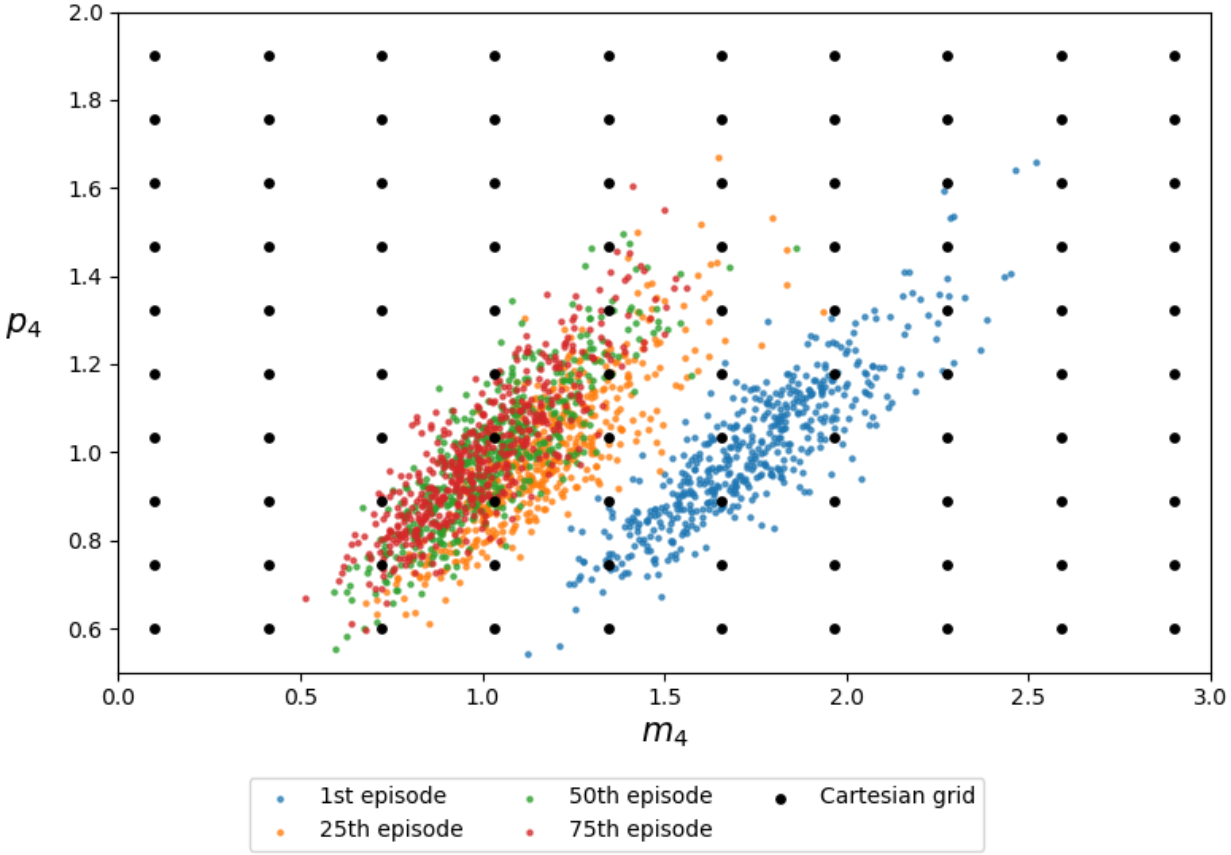


Grids vs simulation: 25th episode



25 episodes → DNN has been trained (a bit) → Cloud is shifting

Grids vs simulation: 75th episode



- Model gets solved where "simulation lives" ("ergodicity")
- Policy approximation is BAD outside (final) "cloud"

Should you always use Deep Learning to solve models?

No.

For many models (including the one today), EGM will be:

- much faster than DeepSimulate
- more numerically stable than DeepSimulate

Advantage of Deep Learning is with large #states a/o #choices

Also, Deep Learning is no *walk-in-the-park*:

- Numerical instability, hyperparameters, ...
- Algorithms are "harder" when we introduce discrete choices (see e.g. DeepVPDDC in Druedahl, Huleux and Røpke 2026)

Ideas for term papers

Term Papers

- Solve high-dimensional model, compare with EGM
- Review DP methods together with more classical Reinforcement Learning methods (Deep Q-learning etc.)
  - Great book for RL is Sutton & Barto: Reinforcement Learning
  - Term paper on absalon does this
- Make parameters of e.g. BufferStock model "pseudo-states" and solve surrogate model, which facilitates fast outer loop in e.g. NFXP
  - Surrogate model = solved model for different parameter values

Papers

- Druedahl and Røpke 2026: *Deep Learning Algorithms for Solving Convex Finite-Horizon Models*
- Druedahl, Huleux and Røpke 2026: *Deep Learning Solutions of Large Non-Convex Life-Cycle Models*
- Hornik et. al. 1989: *Multilayer feedforward networks are universal approximators*